

UVM Design Verification Plan

RISC-V Bit Manipulation Unit



Internship Project

Alaa Armoush

Contents

1	Context	3
1.1	List of Abbreviations	3
2	Introduction	4
2.1	Goals of This Document	4
2.2	Scope of Verification	4
2.3	Assumptions and Constraints	4
3	Design Overview	5
3.1	High-Level Overview	5
3.2	Key Features	5
3.3	Block Diagram	5
3.4	Block Interface	6
4	Verification Strategy	6
4.1	Methodology	6
4.2	Selection of Verification Tests	6
4.3	Specific Corner Cases	7
4.4	Assertions	7
4.5	Coverage Targets	7
5	Testbench Structure	8
5.1	Testbench Architecture	8
5.2	Testbench Customizations	8
5.3	Interfaces	8
6	Verification Test Plan	9
6.1	CSR Read	9
6.2	CSR Write	9
6.3	OR and ORN	9
6.4	XOR and XNOR	10
6.5	Shift Right Logical	11
6.6	Shift Right Arithmetic	11
6.7	Rotate Right	12
6.8	Bit Invert	12
6.9	Shift Left by Two and Add	13
6.10	Subtraction	13

6.11 Set Less Than	13
6.12 Count Trailing Zeros	14
6.13 Population Count	14
6.14 Sign Extend Byte	15
6.15 Signed Maximum	15
6.16 Pack	15
6.17 GREV Byte Reverse	16
6.18 Reset and Temporal Behavior	16
7 Verification Extra Details	17
7.1 Tools and Technologies	17
7.2 Regression Tests	17
7.3 Coverage Reports	17
7.4 Verification Results	17
7.5 Completion and Sign-Off Criteria	17

1 Context

1.1 List of Abbreviations

Abbreviation	Definition
BMU	Bit Manipulation Unit
CSR	Control and Status Register
RISC-V	Reduced Instruction Set Computer, fifth generation
UVM	Universal Verification Methodology
Zba	Address-generation bit-manipulation subset
Zbb	Basic bit-manipulation subset
Zbp	Permutation bit-manipulation subset
Zbs	Single-bit manipulation subset
OR	Bitwise OR
ORN	Bitwise OR with inverted operand B
XOR	Bitwise exclusive OR
XNOR	Bitwise exclusive OR with inverted operand B
SRL	Shift Right Logical
SRA	Shift Right Arithmetic
ROR	Rotate Right
BINV	Bit Invert
SH2ADD	Shift operand A left by two and add operand B
SUB	Subtraction
SLT	Signed Set Less Than
SLTU	Unsigned Set Less Than
CTZ	Count Trailing Zeros
CPOP	Population Count
SEXT.B	Sign Extend Byte
MAX	Signed Maximum
PACK	Pack the lower halfwords of two operands
GREV	Generalized Reverse; used here for byte reversal

2 Introduction

2.1 Goals of This Document

This verification plan defines the BMU functionality to be verified, the required scenarios and corner cases, the planned checking mechanisms, functional coverage, regression content, and completion criteria.

2.2 Scope of Verification

The verification scope contains:

- CSR read and CSR write-data selection;
- OR, ORN, XOR, and XNOR;
- SRL, SRA, ROR, BINV, and SH2ADD;
- subtraction, signed SLT, unsigned SLTU, and signed MAX;
- CTZ, CPOP, and SEXT.B;
- PACK and GREV byte reverse;
- active-low synchronous reset;
- registered-result hold while `valid_in==0`;
- consecutive valid operations; and
- error behavior during valid and idle cycles.

All other operations and alternative parameter configurations are outside the scope of this verification effort.

2.3 Assumptions and Constraints

- The supplied compile-time parameter configuration is used.
- `scan_mode` is tied to zero throughout functional simulation.
- The top-level testbench is the only reset owner.
- Reset is active-low and synchronous. It starts asserted, remains active for at least two rising edges, and is released away from a rising edge.
- Inputs are driven before the rising edge and accepted on that edge when `valid_in==1`.
- One operation may be accepted on each consecutive valid cycle.
- `result_ff` holds its previous value while `valid_in==0`.
- `error` reflects the current control inputs and is not gated by `valid_in`.
- All unrelated decoded-control fields are zero for a legal operation.
- A table-defined invalid operation produces `error==1`. If accepted, its registered result is zero.

3 Design Overview

3.1 High-Level Overview

The BMU is a 32-bit RISC-V execution unit. A packed decoded-control input selects the requested operation. Operands, CSR data, and control fields are processed combinationally, and the functional result is stored in `result_ff`. The `error` output reports invalid control combinations.

3.2 Key Features

- Logical, shift, rotate, single-bit, arithmetic, count, extension, packing, and CSR data-path operations.
- A registered result with one-cycle input-to-result latency.
- One accepted operation per valid cycle.
- Result retention during idle cycles.
- Error detection for invalid operation controls.
- Active-low synchronous reset.

3.3 Block Diagram

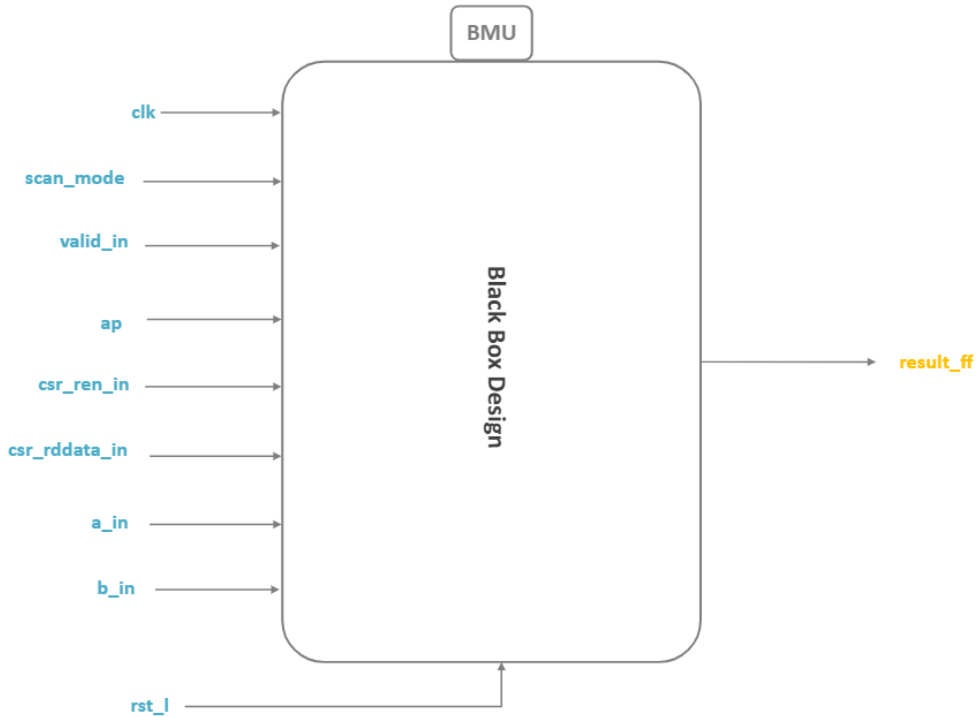


Figure 1: BMU block diagram.

3.4 Block Interface

Signal	Direction	Width/type	Description
clk	Input	1 bit	BMU clock.
rst_l	Input	1 bit	Active-low synchronous reset.
scan_mode	Input	1 bit	Scan control; tied to zero during functional verification.
valid_in	Input	1 bit	Indicates an accepted input operation.
ap	Input	Decoded-control structure	Selects the BMU operation and its required modifiers.
csr_ren_in	Input	1 bit	Enables CSR read/bypass.
csr_rddata_in	Input	32 bits	CSR read data.
a_in	Input	signed 32 bits	Operand A.
b_in	Input	32 bits	Operand B or shift/index value.
result_ff	Output	32 bits	Registered operation result.
error	Output	1 bit	Invalid-control indicator.

4 Verification Strategy

4.1 Methodology

SystemVerilog UVM will be used to generate stimulus, drive the BMU interface, observe requests and responses, calculate expected results independently, compare expected and actual behavior, collect functional coverage, and run repeatable regressions.

The scoreboard associates each accepted request with its one-cycle response. When reset is sampled active, the scoreboard immediately discards every pending prediction before accepting further stimulus.

The monitor publishes a new stable transaction snapshot for every observation. The scoreboard and coverage collector must not retain a transaction handle that can later be modified by the monitor.

4.2 Selection of Verification Tests

Each operation group has a focused sequence containing its basic, boundary, random, invariant, and invalid-control cases. Separate integration tests cover reset, idle result retention, consecutive operations, idle-cycle error, and post-reset recovery.

Constrained-random tests select only in-scope legal operations and explicitly planned invalid combinations. Every randomization call is checked, and every retained failure is reproducible by test name and seed.

4.3 Specific Corner Cases

The test plan covers:

- zero, all ones, alternating patterns, and random values.
- equal and complementary operands.
- shift and bit-index boundaries 0 and 31.
- signed/unsigned comparison differences and signed limits.
- CTZ and CPOP count boundaries.
- ignored upper operand bits and unselected halfwords.
- required mode fields and invalid simultaneous controls.
- reset, idle, back-to-back, error, and recovery behavior.

4.4 Assertions

Assertion	Purpose
A_RST_CLEAR	An active reset sampled on the rising edge clears the registered result.
A_RST_SYNC	Asserting reset between rising edges does not change the registered result before the next rising edge.
A_RESULT_HOLD	When reset is inactive and <code>valid_in==0</code> , <code>result_ff</code> remains stable.
A_KNOWN_INPUTS	Every accepted operation has known controls and operands.
A_ILLEGAL_ERROR	A planned invalid control combination asserts <code>error</code> .
A_IDLE_ERROR	Invalid controls can assert <code>error</code> while <code>valid_in==0</code> without changing <code>result_ff</code> .

Functional calculations remain in the reference model instead of being duplicated in assertions.

4.5 Coverage Targets

Coverage	Target
Functional bins	100% of required bins.
Required crosses	100% of justified crosses.
Assertions	Zero failures in the passing regression, with required properties exercised non-vacuously where applicable.
Code coverage	Every reachable in-scope statement, branch, expression, and meaningful control toggle is covered or reviewed.
Exclusions	Every remaining hole is classified and justified.

Coverage records whether a scenario occurred. The scoreboard determines whether it was correct.

5 Testbench Structure

5.1 Testbench Architecture

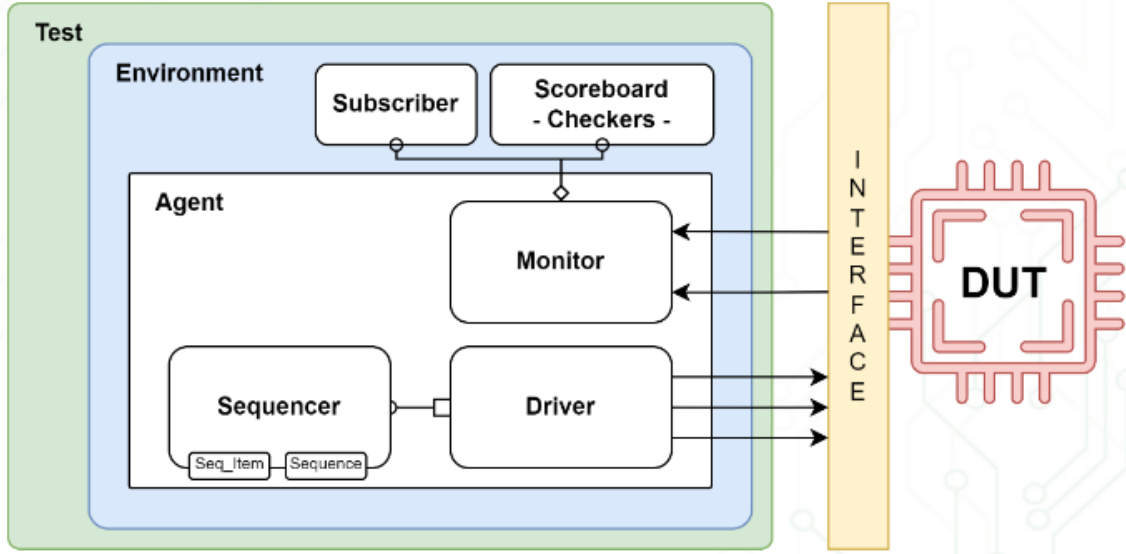


Figure 2: BMU UVM testbench architecture.

5.2 Testbench Customizations

The testbench uses the standard UVM agent–driver–monitor–scoreboard architecture shown in Figure 2. The following BMU-specific implementation choices apply:

- The driver operates on the `ap`, `csr_ren_in`, `a_in`, `b_in`, and `valid_in` signals through a clocking block configured for input setup before the rising edge.
- The monitor samples `result_ff` and `error` on the same clock edge and publishes a complete, immutable transaction snapshot to the scoreboard and coverage collector.
- The scoreboard uses an independent SystemVerilog reference model to predict one-cycle results. On a clock edge where `rst_1` is active, it flushes all pending predictions immediately.
- The environment connects exactly one agent, one scoreboard, and one coverage subscriber. The top-level testbench drives reset and clock; no other component drives reset.
- `scan_mode` is tied to zero in the top-level wrapper and is never randomized or toggled.

5.3 Interfaces

One SystemVerilog interface connects the testbench and DUT. Separate driver and monitor clocking blocks provide race-free drive and sample timing. The virtual interface is configured from the top-level testbench; failure to retrieve it is fatal. Reset is visible through the interface but is driven only by the top-level testbench.

6 Verification Test Plan

Unless a scenario states otherwise, a legal operation has `csr_ren_in==0` and every unrelated decoded-control field cleared. Expected registered results are checked at the operation's one-cycle response. An invalid accepted operation must produce a zero registered result and `error==1`.

6.1 CSR Read

ID	Scenario	Expected behavior
CSR-R-01	Assert <code>csr_ren_in</code> ; clear all decoded controls.	Result equals <code>csr_rddata_in</code> ; error is zero.
CSR-R-02	Use all-zero CSR data.	Result is all zeros.
CSR-R-03	Use all-one CSR data.	Result is all ones.
CSR-R-04	Use random CSR data.	Result equals the sampled CSR data.
CSR-R-05	Assert CSR read with an in-scope BMU operation.	Result is zero; error is one.

Planned stimulus `bmu_csr_read_sequence`

Checker/assertion CSR bypass reference rule and `A_ILLEGAL_ERROR`.

Functional coverage CSR data classes and legal/invalid CSR-read bins.

6.2 CSR Write

ID	Scenario	Expected behavior
CSR-W-01	Set <code>csr_write=1</code> and <code>csr_imm=1</code> .	Result equals <code>b_in</code> ; error is zero.
CSR-W-02	Set <code>csr_write=1</code> and <code>csr_imm=0</code> .	Result equals <code>a_in</code> ; error is zero.
CSR-W-03	Toggle <code>csr_imm</code> using different A and B values.	Result switches to the correct source.
CSR-W-04	Use all zeros on the selected source.	Result is all zeros.
CSR-W-05	Use all ones on the selected source.	Result is all ones.
CSR-W-06	Use random A and B values.	Result equals the selected source.
CSR-W-07	Combine CSR write with CSR read or another in-scope control.	Result is zero; error is one.

Planned stimulus `bmu_csr_write_sequence`

Checker/assertion CSR source-selection reference rule and `A_ILLEGAL_ERROR`.

Functional coverage Immediate mode crossed with source-data class; invalid-control bin.

6.3 OR and ORN

ID	Scenario	Expected behavior
OR-01	Use all-zero operands.	OR produces $A \mid B$; ORN produces $A \mid \sim B$.

ID	Scenario	Expected behavior
OR-02	Use all-one operands.	OR produces $A \mid B$; ORN produces $A \mid \sim B$.
OR-03	Use alternating patterns.	OR produces $A \mid B$; ORN produces $A \mid \sim B$.
OR-04	Use random operands.	Result matches the independent model for the selected mode.
OR-I-01	Use an invalid in-scope control combination (e.g., <code>lor</code> with another operation or CSR read).	Result is zero; error is one.

Planned stimulus `bmu_or_orn_sequence`

Checker/assertion Independent OR/ORN rules and `A_ILLEGAL_ERROR`.

Functional coverage Mode crossed with operand-pattern class; invalid-control bins.

6.4 XOR and XNOR

ID	Scenario	Expected behavior
XOR-01	Use equal operands in both modes.	XOR is zero; XNOR is all ones.
XOR-02	Use complementary operands in both modes.	XOR is all ones; XNOR is zero.
XOR-03	Use all-zero operands.	Result follows the selected XOR/XNOR operation.
XOR-04	Use all-one operands.	Result follows the selected XOR/XNOR operation.
XOR-05	Use random operands.	Result matches the independent model.
XOR-I-01	Use an invalid in-scope control combination.	Result is zero; error is one.

Planned stimulus `bmu_xor_xnor_sequence`

Checker/assertion Independent XOR/XNOR rules and `A_ILLEGAL_ERROR`.

Functional coverage Mode crossed with equal/complementary/data-pattern classes.

6.5 Shift Right Logical

ID	Scenario	Expected behavior
SRL-01	Use valid SRL operands and shift amounts.	Result is $\mathbf{a_in} \gg \mathbf{b_in[4:0]}$.
SRL-02	Shift operands with set MSBs.	Vacated MSB positions contain zeros.
SRL-03	Use an invalid guard combination.	Result is zero; error is one.
SRL-04	Shift by zero.	Result equals $\mathbf{a_in}$.
SRL-05	Shift by 31.	Original bit 31 becomes bit 0; other result bits are zero.
SRL-06	Hold $\mathbf{b_in[4:0]}$ and vary $\mathbf{b_in[31:5]}$.	Shift amount and result do not change.

Planned stimulus $\mathbf{bmu_srl_sequence}$

Checker/assertion Logical-shift rule, zero-fill check, and $\mathbf{A_ILLEGAL_ERROR}$.

Functional coverage Shift amount 0/interior/31 crossed with operand class.

6.6 Shift Right Arithmetic

ID	Scenario	Expected behavior
SRA-01	Use valid SRA operands and shift amounts.	Result is $\mathbf{\$signed(a_in) \gg b_in[4:0]}$.
SRA-02	Shift positive and negative operands.	Vacated MSBs replicate the sign bit.
SRA-03	Use an invalid guard combination.	Result is zero; error is one.
SRA-04	Shift by zero.	Result equals $\mathbf{a_in}$.
SRA-05	Shift by 31.	Result is all zeros or all ones according to the sign bit.
SRA-06	Hold $\mathbf{b_in[4:0]}$ and vary $\mathbf{b_in[31:5]}$.	Shift amount and result do not change.

Planned stimulus $\mathbf{bmu_sra_sequence}$

Checker/assertion Arithmetic-shift rule, sign-fill check, and $\mathbf{A_ILLEGAL_ERROR}$.

Functional coverage Shift amount class crossed with operand sign.

6.7 Rotate Right

ID	Scenario	Expected behavior
ROR-01	Rotate by <code>b_in[4:0]</code> .	Result matches a 32-bit rotate-right.
ROR-02	Use patterns that cross the LSB boundary.	Bits leaving the LSB re-enter at the MSB.
ROR-03	Use an invalid guard combination.	Result is zero; error is one.
ROR-04	Rotate by zero.	Result equals <code>a_in</code> .
ROR-05	Rotate by 31.	Result equals rotate-left by one.
ROR-06	Hold <code>b_in[4:0]</code> and vary <code>b_in[31:5]</code> .	Rotate amount and result do not change.

Planned stimulus `bmu_ror_sequence`

Checker/assertion Independent rotate rule, wrap check, and `A_ILLEGAL_ERROR`.

Functional coverage Rotate amount class crossed with operand pattern.

6.8 Bit Invert

ID	Scenario	Expected behavior
BINV-01	Select an A bit equal to zero.	Selected bit becomes one.
BINV-02	Select an A bit equal to one.	Selected bit becomes zero.
BINV-03	Select bit index zero.	The LSB toggles.
BINV-04	Select bit index 31.	The MSB toggles.
BINV-05	Hold <code>b_in[4:0]</code> and vary <code>b_in[31:5]</code> .	Selected bit and result do not change.
BINV-06	Use representative operands and indices.	Exactly one bit differs between A and the result.
BINV-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus `bmu_binv_sequence`

Checker/assertion One-hot toggle rule, exactly-one-bit check, and `A_ILLEGAL_ERROR`.

Functional coverage Index boundary/interior crossed with selected-bit value.

6.9 Shift Left by Two and Add

ID	Scenario	Expected behavior
SH2-01	Set B to zero.	Result is $A \ll 2$.
SH2-02	Set A to zero.	Result is B.
SH2-03	Use normal and random A/B values.	Result is the low 32 bits of $(A \ll 2) + B$.
SH2-04	Vary <code>b_in[4:0]</code> .	The shift of A remains fixed at two.
SH2-I-01	Assert <code>sh2add</code> with <code>zba=0</code> .	Result is zero; error is one.
SH2-I-02	Assert another in-scope operation control.	Result is zero; error is one.

Planned stimulus `bm_u_sh2add_sequence`

Checker/assertion Independent fixed-shift/add rule and `A_ILLEGAL_ERROR`.

Functional coverage A/B data classes and legal/invalid Zba mode.

6.10 Subtraction

ID	Scenario	Expected behavior
SUB-01	Use A greater than B.	Result is A minus B.
SUB-02	Use equal operands.	Result is zero.
SUB-03	Use A less than B.	Result follows 32-bit subtraction.
SUB-04	Use zero and <code>32'hFFFF_FFFF</code> boundaries.	Result matches the independent 32-bit model.
SUB-I-01	Assert <code>zba=1</code> .	Result is zero; error is one.
SUB-I-02	Assert CSR read or another in-scope control.	Result is zero; error is one.

Planned stimulus `bm_u_sub_sequence`

Checker/assertion Independent modular-subtraction rule and `A_ILLEGAL_ERROR`.

Functional coverage Operand relationship and boundary classes; invalid bins.

6.11 Set Less Than

ID	Scenario	Expected behavior
SLT-01	Use A less than B.	Result is one in the selected mode.
SLT-02	Use A greater than B.	Result is zero in the selected mode.
SLT-03	Use equal operands.	Result is zero.
SLT-04	Reuse operands for which signed and unsigned comparisons differ.	SLT and SLTU produce their respective results.
SLT-05	Compare signed negative and positive values.	Signed ordering is applied.
SLT-06	Compare unsigned large and small values.	Unsigned ordering is applied.
SLT-07	Toggle <code>unsign</code> with the same operands.	Result changes only when signed and unsigned ordering differ.

ID	Scenario	Expected behavior
SLT-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus bmu_slt_sequence

Checker/assertion Independent signed and unsigned comparisons plus A_ILLEGAL_ERROR.

Functional coverage Mode crossed with operand sign and relationship.

6.12 Count Trailing Zeros

ID	Scenario	Expected behavior
CTZ-01	Set the LSB.	Result is zero.
CTZ-02	Use exactly one trailing zero.	Result is one.
CTZ-03	Use 32'h0000_0100.	Result is eight.
CTZ-04	Set only the MSB.	Result is 31.
CTZ-05	Use all zeros.	Result is 32.
CTZ-06	Use random values.	Result matches an independent trailing-zero count.
CTZ-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus bmu_ctz_sequence

Checker/assertion Independent 32-bit trailing-zero model and A_ILLEGAL_ERROR.

Functional coverage Count results 0/1/interior/31/32 and invalid-control bins.

6.13 Population Count

ID	Scenario	Expected behavior
CPOP-01	Use all zeros.	Result is zero.
CPOP-02	Set exactly one bit.	Result is one.
CPOP-03	Use all ones.	Result is 32.
CPOP-04	Use 32'hAAAA_AAAA.	Result is 16.
CPOP-05	Use random values.	Result matches an independent 32-bit population count.
CPOP-06	Place the same number of ones in different positions.	The population result remains the same.
CPOP-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus bmu_cpop_sequence

Checker/assertion Independent 32-bit population model and A_ILLEGAL_ERROR.

Functional coverage Count results 0/1/16/interior/32 and position classes.

6.14 Sign Extend Byte

ID	Scenario	Expected behavior
SEXB-01	Use a low byte with bit 7 equal to zero.	Upper 24 result bits are zero.
SEXB-02	Use a low byte with bit 7 equal to one.	Upper 24 result bits are one.
SEXB-03	Hold <code>a_in[7:0]</code> and vary <code>a_in[31:8]</code> .	Result does not change.
SEXB-04	Use random low-byte values.	Result matches independent sign extension.
SEXB-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus `bmu_sext_b_sequence`

Checker/assertion Independent sign-extension rule and `A_ILLEGAL_ERROR`.

Functional coverage Low-byte sign crossed with data class; upper-A invariance.

6.15 Signed Maximum

ID	Scenario	Expected behavior
MAX-01	Use signed A greater than B.	Result is A.
MAX-02	Use signed A less than B.	Result is B.
MAX-03	Use equal operands.	Result is that operand value.
MAX-04	Compare a negative value with a positive value.	The positive value is returned.
MAX-05	Compare two negative values.	The less-negative value is returned.
MAX-06	Compare <code>32'h8000_0000</code> with <code>32'h7FFF_FFFF</code> .	The signed maximum is returned.
MAX-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus `bmu_max_sequence`

Checker/assertion Independent signed-maximum rule and `A_ILLEGAL_ERROR`.

Functional coverage Signed relationship crossed with sign and boundary class.

6.16 Pack

ID	Scenario	Expected behavior
PACK-01	Use different lower halves.	Result is <code>{B[15:0], A[15:0]}</code> .
PACK-02	Change only <code>A[31:16]</code> .	Result does not change.
PACK-03	Change only <code>B[31:16]</code> .	Result does not change.
PACK-04	Use all zeros and all ones in the selected halves.	Result contains the selected lower halves.
PACK-05	Use random A and B values.	Result matches independent concatenation.
PACK-I-01	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus `bmu_pack_sequence`

Checker/assertion Independent lower-half concatenation and invariance checks.

Functional coverage Selected-half patterns, upper-half invariance, and invalid bins.

6.17 GREV Byte Reverse

ID	Scenario	Expected behavior
GREV-01	Set <code>b_in[4:0]=24</code> .	The four A bytes reverse order.
GREV-02	Use <code>a_in=32'h1234_5678</code> .	Result is <code>32'h7856_3412</code> .
GREV-03	Use an alternating byte pattern.	Byte positions match the independent reversal.
GREV-04	Use random A values with mode 24.	Result matches independent byte reversal.
GREV-I-01	Use <code>b_in[4:0] != 24</code> .	Result is zero; error is one.
GREV-I-02	Use another in-scope control or CSR read.	Result is zero; error is one.

Planned stimulus `bmu_grev_sequence`

Checker/assertion Independent byte-reversal rule, mode check, and `A_ILLEGAL_ERROR`.

Functional coverage Mode 24/invalid crossed with byte-pattern class.

6.18 Reset and Temporal Behavior

ID	Scenario	Expected behavior
TMP-RST-01	Hold reset active across at least two rising edges.	Registered result and error are zero on active reset edges.
TMP-RST-02	Assert reset between rising edges after creating a nonzero result.	The registered result changes only on the reset clock edge.
TMP-HOLD-01	Set <code>valid_in=0</code> and vary operands/controls.	<code>result_ff</code> retains its previous value.
TMP-B2B-01	Apply different legal operations on consecutive valid cycles.	Responses remain ordered without loss or duplication.
TMP-IDLE-ERR-01	Apply an invalid in-scope combination while <code>valid_in=0</code> .	Error asserts while <code>result_ff</code> holds.
TMP-REC-01	Release reset and accept a legal operation.	The first post-reset result is correct and error-free.
TMP-FLUSH-01	Assert reset while a prediction is pending.	The scoreboard discards all pending predictions on the reset edge.

Planned stimulus `bmu_reset_test`, `bmu_valid_hold_test`, and `bmu_back_to_back_test`

Checker/assertion Reset/hold/alignment rules, `A_RST_CLEAR`, `A_RST_SYNC`, `A_RESULT_HOLD`, and `A_IDLE_ERROR`.

Functional coverage Reset/recovery, idle hold, operation transitions, and idle-error crosses.

7 Verification Extra Details

7.1 Tools and Technologies

- SystemVerilog and UVM;
- Cadence Xcelium `xrun(64)` 25.03-s006;
- GNU Make;
- SimVision and IMC when waveform or coverage analysis is needed; and
- Verisium as an interactive or post-process debugging aid.

The project will provide a `make check` target that compiles and elaborates the complete current testbench without running a feature test.

7.2 Regression Tests

The regression contains every feature sequence, all reset and temporal tests, reproducible constrained-random runs, and a focused reproducer for each confirmed DUT bug. Passing tests and expected DUT failures are reported separately. Every complete test has a timeout and an automatic PASS/FAIL verdict.

7.3 Coverage Reports

Coverage reports record functional and code coverage, uncovered required scenarios, reviewed exclusions, and whether each covered scenario passed its checker.

7.4 Verification Results

Execution records contain the test name, sequence, seed, parameter configuration, simulator version, verdict, mismatch count, confirmed bugs, and coverage contribution. A final report summarizes these results without changing this verification contract.

7.5 Completion and Sign-Off Criteria

Verification is complete when:

1. every required scenario has implemented stimulus and automatic checking;
2. the clean compile/elaboration and infrastructure sanity flow succeeds;
3. all required passing tests complete without unexplained errors, mismatches, or timeouts;
4. confirmed DUT failures have focused reproducible tests;
5. required functional bins and justified crosses reach 100%;
6. every in-scope code-coverage hole is closed or justified;
7. required assertions pass and are exercised meaningfully;
8. exact tests, seeds, configuration, simulator version, results, and limitations are recorded; and
9. the verification results demonstrate every requirement in this plan.